

HyperEye: A Lightweight Features Fusion Model for Unknown Encrypted Malware Traffic Detection

Xiaodong Zang[✉], Zilong Zheng, Haosheng Zheng, Xuan Liu[✉], *Senior Member, IEEE*,
Muhammad Khurram Khan[✉], *Senior Member, IEEE*, and Weiwei Jiang[✉]

Abstract—Recently, effectively identifying encrypted malicious traffic without decryption in consumer applications relies heavily on high-quality labeled traffic datasets. However, this harms models for incorrect labeling and requires more efficient real-time identification of encrypted unknown ones. This paper proposes HyperEye, a real-time, unsupervised, encrypted malicious traffic detection system. It can detect unknown traffic patterns by analyzing the fused traffic features in-depth. Precisely, we extract protocol-agnostic numerical and protocol-specific text features and devise a cross-term fusion algorithm to obtain a comprehensive traffic behavior description. We designed a genetic algorithm-based DBSCAN (GA-DBSCAN) parameter optimization algorithm to enhance the quality and stability in identifying malicious traffic. Extensive experimental results with open-world and real-world datasets demonstrate that our work outperforms other state-of-the-art malware detection systems, achieving an average 11.95% improvement in the F1-score. Besides, experimental results with the real-world dataset demonstrate that our system applies to the dynamic nature of consumer applications and can safeguard users' data and privacy.

Index Terms—Malicious traffic, consumer applications, NetFlow, features fusion, threat detection.

I. INTRODUCTION

WITH the increase of new networking paradigms, e.g., the Internet of Things (IoT), the Internet of Vehicles (IoV), and 5G and 6G cellular networks, Internet traffic has grown exponentially [1]. In order to ensure information security and data privacy, encrypted traffic protocols have been widely adopted, e.g., (SSL/TLS and HTTPS) [2]. However, encryption-based techniques for such protection are double-edged, as the attackers can also use them to conceal their malicious behaviors, e.g., C&C communication. Therefore,

identifying malicious behavior accurately and quickly from massive encryption traffic in consumer applications is one of the keys to defending against cyber attacks [3], [4].

Many previous works applied machine learning, knowledge-based, and graph-based analysis methods for encrypted malicious detection [1], [3]. The knowledge-based methods used prior knowledge to accomplish traffic analysis tasks. The machine learning-based methods need complex feature engineering to obtain manually crafted features as input [5]. Both are inefficient in identifying unknown encrypted malicious traffic due to the need for prior knowledge of known attacks. The graph-based methods [6], [7], [8] are proposed to capture the flow interaction patterns. However, the graph structure built upon the traffic patterns requires much time, which makes it challenging to build a graph for real-time detection.

After investigating these works, we find two challenges that must be tackled: (1) *Real-time detection*, (2) *Unknown malicious traffic identification*. We analyze the packet header information and cooperate with existing on-path monitor tools to throttle the detected malicious one after identifying it. We focus on identifying active attacks caused by attackers without detecting the passive ones (e.g., eavesdropping and intercept attacks) as they do not cause apparent traffic variance in the backbone.

To achieve real-time detection, we devise a cross-term fusion algorithm by analyzing the protocol-agnostic numerical and protocol-specific text features in-depth to capture the behavioral footprints comprehensively. We convert the protocol-agnostic numerical features into traffic maps and use 2D-CNN to extract the traffic information to obtain the high-level representation corresponding to the image. For the protocol-specific features, we use TF-IDF to measure the importance of the extracted text features and transform protocol-specific features into vector representations. To achieve unknown malicious traffic identification, we designed a genetic algorithm-based DBSCAN parameter optimization method to find the optimal parameter combination in ample parameter space automatically. It reduces the complexity of artificial parameter adjustment and improves the quality and stability of identifying malicious traffic samples.

We conduct extensive experiments on two open-world and two real-world datasets to evaluate the generalization ability. The main contributions of this paper are as follows:

- (1) We devised a novel feature fusion model by analyzing the protocol-agnostic numerical features and protocol-specific text features in-depth to obtain a comprehensive behavior description of traffic behavior.

Received 8 December 2024; revised 9 February 2025; accepted 29 March 2025. Date of publication 8 April 2025; date of current version 14 August 2025. This work was supported in part by the King Saud University, Riyadh, Saudi Arabia, under Project RSP2025R12; in part by the Yangzhou Science and Technology Plan Project under Grant YZ2023200; and in part by the Self-Developing Experimental Instrument and Equipment Project of Yangzhou University under Grant zzyq2023zy06. (Corresponding authors: Haosheng Zheng; Xiaodong Zang.)

Xiaodong Zang, Zilong Zheng, and Haosheng Zheng are with the School of Cyber Science and Engineering, Qufu Normal University, Qufu 273165, China (e-mail: xdzang@qfnu.edu.cn; hszheng815@163.com).

Xuan Liu is with the College of Information Engineering (College of Artificial Intelligence), Yangzhou University, Yangzhou 225127, China.

Muhammad Khurram Khan is with the Center of Excellence in Information Assurance, King Saud University, Riyadh 11653, Saudi Arabia.

Weiwei Jiang is with the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China.

Digital Object Identifier 10.1109/TCE.2025.3558353

- (2) We designed a genetic algorithm-based DBSCAN method to enhance the quality and stability of identifying malicious traffic samples.
- (3) We conducted extensive evaluations and found that our system's overall detection rate is 99.8%, and its false alarm rate is less than 0.05%, outperforming other state-of-the-art alternatives. We also applied it to IoT malicious detection with State Grid traffic to verify that it is suitable for the edge of the IoT environment.

Organization: In Section II, we survey the malware traffic detection methods and give a detailed comparison. Section III elaborates on our methodology. In Section IV, we conduct extensive experiments with different datasets to verify the superiority of our approach. Finally, we present the conclusion.

II. RELATED WORKS

In this section, we survey encrypted malicious traffic detection works and broadly classify them into knowledge, traditional machine learning, deep learning, and graph-based detection methods.

A. Knowledge-Based Detection Methods

Knowledge-based detection methods contain the knowledge base and the inference engine. They identify the malicious traffic with prior knowledge. In the last few years, researchers have used pre-established rules or similarity analysis [10] to achieve traffic classification or malicious traffic identification. For example, [10] proposed a maliciously encrypted traffic detection framework with a causality explainable inference engine. Their experimental results provide causal traceability for malicious encrypted traffic. However, the completeness of the knowledge base requires regular updates.

B. Traditional Machine Learning-Based Detection Methods

Traditional machine learning techniques are essentially shallow learning. They require manual feature extraction and selection [18], [19]. Reference [20] designed a tree-based algorithm to identify malicious traffic with the extracted statistical features, such as flow length and packet length. Experimental results show that its detection rate reaches 90%. Reference [11] used machine-learning models that could rely on human expertise to extract valuable and informative features and identify encrypted malicious HTTP traffic. Although traditional machine-learning techniques have high detection accuracy, they must obtain complete statistical characteristics by collecting long-term streams, limiting their real-time detection and generalization abilities.

C. Deep Learning-Based Detection Methods

Deep learning methods have recently been widely used in encrypted traffic identification without decryption. Deep learning techniques have automatic feature extraction ability, which enables raw traffic representation as input [14], [21], [22], [23]. For example, Lin et al. [15] devised a feature fusion model that combines data timing, byte, and other statistical features as valid information input. EETC [24] transformed the

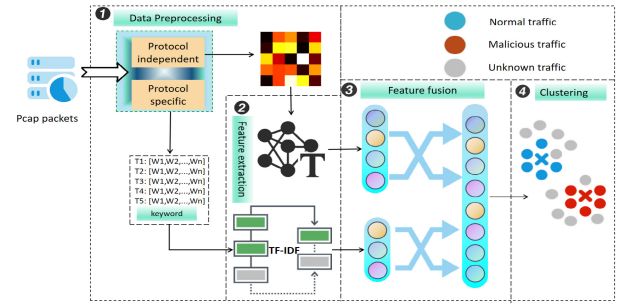


Fig. 1. Methodology of Flow Diagram.

extracted content and statistical information into three-channel RGB images from encrypted traffic. Although it benefits from the advantage of deep learning for encrypted traffic analysis, it is an open issue of improving its interpretability and understanding of the reasoning in anomaly detection [16].

D. Graph-Based Detection Methods

With increasing attention to the traffic flow interaction patterns of encrypted traffic, graph structures are applied to represent encrypted traffic data and devised different graph algorithms for encrypted malicious traffic detection [6], [7], [8]. For example, [6] designed a real-time unsupervised malicious traffic detection system by analyzing the graph's connectivity, sparsity, and statistical features. Hong et al. [8] presented MalDiscovery by constructing an attribute KNN graph. However, a more significant number of graph edges brings a higher time cost, which makes it challenging to deploy in large enterprise network environments with high bandwidth and realize online detection.

We survey the state-of-the-art malware traffic detection methods and compare them in Table I, where Y indicates the corresponding column meets the conditions, N is unsatisfied, and P is partially satisfied.

III. FRAMEWORK OVERVIEW

In this section, we introduce HyperEye, a lightweight model for unknown encrypted malware traffic detection, which mainly contains four parts: Data processing, Feature extraction, Feature fusion, and Traffic clustering, as illustrated in Fig. 1.

A. Data Processing

Our original real traffic is pcap files. We clean data to remove incomplete, invalid, or unnecessary packets, such as traffic with no payload. Additionally, when processing industrial data transmitted from the edge tier, there are often consistency issues (e.g., signal loss or deviations in sensor readings at specific points). We clean the industrial data by eliminating infinite and 'NaN' values to address this. We use randomness-testing for the unknown encrypted traffic to assess their randomness and diversity [25]. The DNS traffic is not filtered, as malware masters always use it to conceal their C&C.

TABLE I
COMPARISONS OF THE RELATED WORKS

Categories	Data Sources	Key technology	Data for Detection		Design Goals			Detection Performance	
			Unlabeled Datasets	Multi - Dimension Features	Generic Detection	Realtime Detection	Unknown Attacks	Low Latency	High Throughput
Knowledge - Based Methods	Causal Features	Wasserstein generative adversarial network[9]	<i>N</i>	<i>N</i>	<i>N</i>	<i>N</i>	<i>P</i>	<i>N</i>	<i>P</i>
	SSL/TLS fingerprinting	Features dictionary [10]	<i>Y</i>	<i>Y</i>	<i>N</i>	<i>Y</i>	<i>Y</i>	<i>Y</i>	<i>N</i>
	HTTPS Headers	SVM and Random forest[11]	<i>N</i>	<i>N</i>	<i>N</i>	<i>N</i>	<i>P</i>	<i>P</i>	<i>N</i>
Traditional ML - Based Methods	Flow Statistics	Random Forest and XGBoost[12]	<i>N</i>	<i>Y</i>	<i>N</i>	<i>N</i>	<i>N</i>	<i>N</i>	<i>N</i>
	IDS Logs	Hidden Markov Model[13]	<i>N</i>	<i>N</i>	<i>N</i>	<i>Y</i>	<i>Y</i>	<i>N</i>	<i>N</i>
	Packet	RF, CNN and GRU[5]	<i>Y</i>	<i>N</i>	<i>N</i>	<i>N</i>	<i>Y</i>	<i>N</i>	<i>N</i>
Deep Learning - Based Methods	biflow	Multitask and multimodal[14]	<i>N</i>	<i>N</i>	<i>Y</i>	<i>N</i>	<i>P</i>	<i>P</i>	<i>N</i>
	Flow Statistics	Features Fusion[15]	<i>N</i>	<i>Y</i>	<i>N</i>	<i>P</i>	<i>P</i>	<i>N</i>	<i>N</i>
	Flow Temporal Features	WF - Transformer[16]	<i>N</i>	<i>N</i>	<i>N</i>	<i>Y</i>	<i>Y</i>	<i>Y</i>	<i>N</i>
	Flow Statistics	CNNs[17]	<i>N</i>	<i>N</i>	<i>P</i>	<i>N</i>	<i>P</i>	<i>N</i>	<i>N</i>
	Multi - view Features	KNN graph[8]	<i>Y</i>	<i>Y</i>	<i>N</i>	<i>N</i>	<i>Y</i>	<i>N</i>	<i>P</i>
	Flow Statistical Features	Flow Interaction Graph [6]	<i>Y</i>	<i>N</i>	<i>N</i>	<i>Y</i>	<i>Y</i>	<i>Y</i>	<i>Y</i>
Graph - Based Methods	Spatial and Temporal Characteristics	ST - Graph [7]	<i>Y</i>	<i>N</i>	<i>N</i>	<i>Y</i>	<i>Y</i>	<i>Y</i>	<i>P</i>
	Multi - view Features	Features Fusion	<i>Y</i>	<i>Y</i>	<i>P</i>	<i>Y</i>	<i>Y</i>	<i>Y</i>	<i>Y</i>

TABLE II
PROTOCOL-AGNOSTIC FEATURES

No.	Feature	Description
1	FlowDur	Duration of the flow
2	FwdDur	Duration of forward traffic
3	BwdDur	Duration of backward traffic
4	MeanTime	Mean packet time difference per session
5	StdTime	Std. deviation of packet time difference
6	MeanIATFwd	Mean interval of forward packet arrivals
7	MeanIATBwd	Mean interval of backward packet arrivals
8	StdIATFwd	Std. deviation of forward packet intervals
9	StdIATBwd	Std. deviation of backward packet intervals
10	FwdIPLenTot	Total forward IP header length
11	BwdIPLenTot	Total backward IP header length
12	MeanFwdIPLen	Mean forward IP header length
13	MeanBwdIPLen	Mean backward IP header length
14	FwdTCPLenTot	Total forward TCP header length
15	BwdTCPLenTot	Total backward TCP header length
16	PldTot	Total payload length per session
17	FwdPldTot	Total forward payload length
18	BwdPldTot	Total backward payload length
19	MeanTCPLen	Mean TCP payload length
20	StdTCPLen	Std. deviation of TCP payload length
21	StdFwdPktLen	Std. deviation of forward packet length
22	StdBwdPktLen	Std. deviation of backward packet length
23	MeanIPLen	Mean IP packet length
24	StdIPLen	Std. deviation of IP packet length
25	IP Ratio	Max/min IP packet length ratio
26	Pld Ratio	Payload length ratio
27	Goodput	IP packet length per session / flow duration
28	MeanTCPWinFwd	Mean TCP window size of forward traffic
29	MeanTCPWinBwd	Mean TCP window size of backward traffic
30	TotTTLFwd	Total TTL of forward traffic
31	TotTTLBwd	Total TTL of backward traffic
32	MeanTTLFwd	Mean TTL of forward traffic
33	MeanTTLBwd	Mean TTL of backward traffic

B. Feature Extraction

Each pcap file corresponds to a specific application, a traffic category, or an encryption technique. In the pcaps, there are TCP-based features, UDP (QUIC, HTTP/3) based and TLS 1.3. We split each pcap file to unidirectional a 5-tuple flow (source IP, source port, destination IP, destination port,

TABLE III
PROTOCOL-SPECIFIC FEATURES

No.	Feature	Description
1	PldBCLients	Total payload bytes from client-side traffic
2	RRCLients	Ratio of responder traffic to client traffic
3	NoCert	Number of certificates
4	TLSVerType	Types of TLS versions observed
5	SNISAN	SNI field presence in SAN DNS
6	DiffSNISSL	Variation of SNI across SSL records
7	DiffSubjSSL	Variation of Subject across SSL records
8	MCValidity	Average validity period of certificates
9	CNameSAN	Common Name field in SAN DNS
10	NoDSAN	Number of domains within SAN
11	CipherSuites	Offered ciphersuites list
12	AdvExtensions	List of advertised extensions
13	PubKeyLen	Length of public key
14	AvgKeyLen	Average public key length
15	TLSRatio	Ratio of TLS to non-TLS records
16	SSignedCert	Indicator if server certificate is self-signed
17	SSLtoNSSLR	Ratio of SSL to non-SSL traffic records
18	SNIRatioSSL	Proportion of SSL records with SNI field
19	SSRatio	Proportion of traffic with self-signed certificates
20	DSNIRatio	Ratio of SSL records with varying SNI
21	DSubjRatio	Ratio of SSL records with varying subjects
22	DIssuerRatio	Ratio of SSL records with different issuers
23	SSubjRatio	Ratio of SSL records with identical subjects
24	SISsuerRatio	Ratio of SSL records with identical issuers
25	TLDSNI	Indicator if SNI is a top-level domain

protocol). We initially quantified the differences in features by calculating the mean and standard deviations between normal and malicious traffic and subsequently removed features with low variance. We utilize the Gradient Boosting Feature Selection (GBFS) algorithm to experimentally select the top 33 protocol-agnostic numerical features and the top 25 protocol-related features apart from five-tuple features, as shown in Tables II and III. We put the extracted protocol-agnostic numerical features (33 dimensions) in an IP flow and transform these protocol-agnostic numerical features into images. Every numerical feature becomes a pixel in the image.

1) *CNN-Based Feature Extraction*: We convert the numerical features to traffic maps and use 2D-CNN to capture its latent relationships. We introduce flow-slicing and flow cleaning to ensure the consistency and purity of the data.

(1) Flow slicing. We collected the real traffic at CERNET. The timestamp is attached to the metadata at the time of its collection. Namely, the timestamp information of the IP traffic is located within the five-minute collection period. Considering that a complete network traffic session contains a large amount of data, it may be practical to process only some of it directly. Therefore, each network traffic session is sliced according to a fixed time window, and each slice is treated as an independent processing unit. This step helps break up long sessions into smaller pieces that are easier to manage and handle.

$$\text{Slices} = \text{Slice}(\text{Session}, \text{Window size}) \quad (1)$$

where *Session* represents a network traffic session, *Window size* is a pre-defined time window size.

(2) Flow cleaning. Traffic cleaning is done after traffic slicing to remove redundant and irrelevant information that may affect the classification effect, including removing protocol header non-payload and potentially noisy data from traffic data.

$$\text{Cleaned} - \text{Slices} = \text{Clean}(\text{Slices}) \quad (2)$$

We then transform the numerical features into grayscale images to better learn the correlation between the features. However, generating grayscale maps can result in a loss of information. We adopt the hash function method to reduce this loss, as it has good collision resistance.

We use the 2D-CNN model to extract features from each processed image as it has been frequently used for two-dimensional signal processing, such as image recognition [26]. Assuming that the convolution kernel of the input $5*5*3$ gray-scale images is W , the deviation is b , and the step size is s , the output calculation formula for each position is as follows:

$$F_{ij} = \sigma \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \sum_{d=0}^{c-1} I_{(i*s+u), (j*s+v), d} * W_{u,v,d} + b \quad (3)$$

where F_{ij} is the value at position (i, j) in the output feature map of the convolution layer. σ is an activation function, such as ReLU. I is the input grayscale images. k is the size of the convolution kernel, and c is the number of channels of the input grayscale images.

After each layer of convolution, a pooling layer is added. To better distinguish the similarity of different protocols, we adopt maximum pooling to help improve the most significant features and add Dropout layers to prevent overfitting. We chose maximum pooling because it more effectively retains key features while ignoring irrelevant background noise. The calculation formula for maximum pooling is shown in Equation (4).

$$O_{ij} = \max_{u,v} (I_{(i*s+u), (j*s+v)}) \quad (4)$$

where O_{ij} is the value at position (i, j) in the output image of the maximum pooling layer, namely, the value at this

position in the image obtained after pooling. (u, v) represents the index in the pooled window, which is used to traverse the elements in the pooled window. The input image is the output of the previous convolutional layer. (i, j) indicates the starting position in the input image for the region to which the maximum pooling operation is applied. s is the step length of the pooled window, which represents the distance of each slide of the pooled window. We use the flatten layer to transform multidimensional inputs X into one-dimensional inputs and use the fully connected layers to learn the nonlinear dependencies of the data.

$$Z = W * X + b \quad (5)$$

where Z is the output of the fully connected layer, W is the weight matrix. X is the input vector (the flattened eigenvector), and b is the offset.

2) *TF-IDF-Based Feature Extraction*: Traffic packets typically contain IP address, port number, MAC address, handshake information, and other fields. Some fields may directly affect the detection algorithms' accuracy, while others do not. After the feature engineering on the traffic data, the digital form characteristics are input for the machine learning model. As the extracted protocol-specific text features are not quickly processed for the general machine learning method, we use the TF-IDF model to transform the text features into digital vectors.

TF-IDF is a commonly used method for feature extraction of text data [27]. The main idea is that if a word has a high frequency (TF) in the current document and a low frequency (IDF) in the whole corpus, it is regarded as distinguishable and suitable for text classification tasks. TF represents the frequency of keywords appearing in the text, and IDF represents reverse file frequency. The calculation methods for TF and IDF are as follows.

$$TF_{ij} = \frac{n_{ij}}{\sum_k n_{kj}} \quad (6)$$

where n_{ij} is the number of occurrences of the word i in the file j , and the denominator is the sum of occurrences of all the words in the file j .

$$IDF(x) = \log \frac{N}{N(x)} \quad (7)$$

where N represents the total number of texts in the corpus, and $N(x)$ represents the total number of texts containing the word x in the corpus.

We use the TF-IDF model to transform the extracted top 25 text features into digital forms. To simplify, assuming that the sampled dataset has 5,842 encrypted traffic data. After converting, the resulting data set is a matrix of $5,842*25$, Fig. 2 shows the calculation process.

C. Feature Fusion

In the last section, we use 2D-CNN and TF-IDF to vectorize the original traffic characteristics with different perspectives.

We design a cross-term fusion algorithm to obtain more effective abstract features and their correlation. We enhance the expressive ability of the model by combining cross-generating new features to capture interactive information

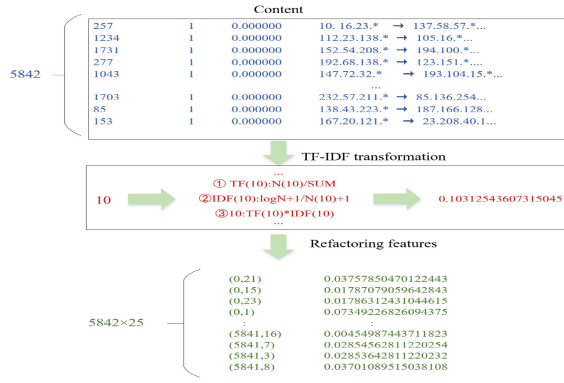


Fig. 2. Text information and reconstruct feature based on TF-IDF.

Algorithm 1 The Pseudocode of Cross-Term Fusion Algorithm

Input: $\text{cnn_vector} = \{v_1, v_2, v_3, \dots, v_n\}$
 $\text{tfidf_vector} = \{v_1, v_2, v_3, \dots, v_m\}$
Output: $\text{merged_vector} = \text{cross_features} + \text{tfidf_vector}$

```

1:  $\text{cross\_features} = []$ 
2: for  $i = 1$  to  $\text{len}(\text{cnn\_vector})$  do
3:    $\text{cross\_feature} = []$ 
4:   if  $i > 0$  then
5:     for  $j = 1$  to  $\text{len}(\text{tfidf\_vector})$  do
6:        $t\_feature = \text{cnn\_vector}[i] \times \text{tfidf\_vector}[j]$ 
7:        $\text{cross\_feature.append}(t\_feature)$ 
8:     end for
9:      $\text{cross\_features.append}(\text{Max}(\text{cross\_feature}))$ 
10:  else
11:    for  $j = 1$  to  $\text{len}(\text{tfidf\_vector})$  do
12:       $t\_feature = \text{cnn\_vector}[i] + \text{tfidf\_vector}[j]$ 
13:       $\text{cross\_feature.append}(t\_feature)$ 
14:    end for
15:     $\text{cross\_features.append}(\text{Min}(\text{cross\_feature}))$ 
16:  end if
17: end for
18: return  $\text{merged\_vector}$ 

```

between features and promote the model to learn the correlation and nonlinear relationship between them.

Our cross-term fusion algorithm constructs a multi-dimensional representation of network behavior. It comprehensively considers the traffic overview reflected by numerical features and the protocol details in text features. Through cross-term features, the algorithm can trace the impact of original features on the model output, identify key factors, and then clearly demonstrate the models decision-making process by visualizing the weights of cross-term features, thus enhancing interpretability. Comprising feature concatenation algorithm [15], it does not consider the mutual influence or correlation between features. The pseudocode of the cross-term fusion algorithm is shown as follows.

D. Traffic Clustering

In this section, we enhance an unsupervised learning method to tackle the limitation of detecting unknown encrypted malicious traffic in real-time.

We employ the DBSCAN algorithm to capture the similarity of fused traffic features to achieve unknown traffic identification in an unsupervised learning way. We chose the DBSCAN algorithm because it does not need to determine how many unknown protocols are in real traffic. However, in the real-world network, the data changes dynamically with time, and it is challenging to select the initial parameter of neighborhood radius “eps” and the minimum core number “MinPts” of DBSCAN. We designed a genetic algorithm-based DBSCAN (GA-DBSCAN) parameter optimization method to address this challenge. Our optimized DBSCAN algorithm contains three steps.

Firstly, we initialize the population. The population consists of several individuals, each representing a set of DBSCAN parameter combinations, namely “eps” and “MinPts.” The initialization ranges for “eps” and “MinPts” in the genetic algorithm were determined by analyzing the dataset’s statistical characteristics. The range for “eps” was set by examining the distribution of distances between points in the dataset, ensuring that the values captured typical clustering densities without being overly restrictive or too loose. For “MinPts”, we considered the average density of points in the dataset to ensure that the genetic algorithm could effectively explore the parameter space, converge efficiently, and produce robust clustering results.

Then, we designed the fitness function to evaluate the merits of each individual (i.e., each set of DBSCAN parameters). We design a multi-objective fitness function $f(x_i)$ using intra-cluster tightness (f_1), inter-cluster separation (f_2), and contour coefficient (f_3) to measure the clustering effect more comprehensively. Intra-cluster tightness measures the tightness of samples within a cluster, inter-cluster separation measures the separation degree between different clusters, and contour coefficient comprehensively measures the tightness of points within a cluster and the separation degree between clusters. The equations are as follows.

$$f_1 = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\| \quad (8)$$

where, C_i is the i th cluster, and μ_i is the center of it.

$$f_2 = \min_{i \neq j} \|\mu_i - \mu_j\| \quad (9)$$

where, μ_i and μ_j is the center of cluster i and cluster j , respectively.

$$f_3 = \frac{(b_i - a_i)}{\max(a_i, b_i)} \quad (10)$$

where, a_i represents the average distance between sample i and other samples in its cluster, and b_i represents the average distance between sample i and the sample in the nearest cluster.

We combine the above indicators in a weighted way, assuming that the weight of each indicator is w_1 , w_2 , and w_3 ,

and the summation of w_1 , w_2 , and w_3 is one, respectively. We get the final fitness function $f(x_i)$ as follows.

$$f_{x_i} = w_1 * f_1 + w_2 * f_2 + w_3 * f_3 \quad (11)$$

Thirdly, we use selection, crossover, and variation to optimize the population from generation to generation. The selection operation is used to select individuals from the current population as parents to generate the next generation of individuals. Assuming the fitness of the i th individual is f_{x_i} , its probability of being selected is P_i .

$$P_i = \frac{f_{x_i}}{\sum_{j=1}^n f_{x_j}} \quad (12)$$

We use cross-operation to combine the genes of two parents to produce a new offspring. For DBSCAN parameters “eps” and “MinPts”, we use a single-point crossover approach. If the parent is $x_1=(eps_1, MinPts_1)$ and $x_2=(eps_2, MinPts_2)$, then the two children are $y_1=(eps_1, MinPts_2)$, $y_2=(eps_2, MinPts_1)$.

We use mutation operations to randomly adjust the gene values of the offspring to prevent excessive perturbation. Assuming that the offspring is $y_i=(eps_i, MinPts_i)$, and the mutation operation can randomly change its parameter values, $eps_i=eps_i + \Delta eps$, $MinPts_i=MinPts_i + \Delta MinPts$, where Δeps and $\Delta MinPts$ are small random perturbations, preventing the algorithm from falling into local optimal solutions. The pseudo-code of its optimization is shown in Algorithm 2.

IV. EVALUATIONS

We conduct our experiments on an Intel Xeon E5-2680 v4 CPU with a memory size of 64GB. We use open-source datasets (CICIDS 2017, CICDDoS 2019) and real-world traffic (CERNET, State Grid) to verify the performance and scalability of our work.

A. Datasets and Evaluation Metrics

CICIDS 2017. University of New Brunswick released the open-source datasets CICIDS 2017 and CICDDoS 2019 to validate the performance of the designed intrusion detection algorithms and research on the application of machine learning algorithms in network traffic analysis and anomaly detection. The dataset contains Brute Force FTP, Brute Force SSH, DoS, Heartbleed, Web Attack, Infiltration, Botnet, and DDoS. We randomly selected 1.98GB of traffic in the experiment.

CICDDoS 2019. The CIC-DDoS 2019 dataset contains benign traffic samples and the latest common DDoS attack samples, covering reflective DDoS attack types such as PortMap, MSSQL, UDP, SYN, NTP, etc. We collected 2.67GB of network traffic and combined it with the detection results of the tag-based network traffic analysis tool (CICFlowMeter-V3).

The CERNET traffic. We collected the traffic at CERNET from 10/1/2024 to 10/7/2024 [28]. During one week, we have captured 4.12 TB packets. After preprocessing, we obtained 50.12 GB encrypted traffic, e.g., TLS traffic and Https traffic, for the experiment. The topology of CERNET can be found on Fig. 3.

Algorithm 2 The Pseudo-Code of Optimized DBSCAN Based on Genetic Algorithm

Input: Dataset X , $[eps_{min}, eps_{max}]$, $[MinPts_{min}, MinPts_{max}]$, $Generations_{max} G$, Mutation rate r_{mut} , Weights w_1, w_2, w_3

Output: $eps_{best}, MinPts_{best}$

- 1: **Initialize** population P of size N with random individuals ($eps, MinPts$)
- 2: **Initialize** an empty list to store fitness scores S
- 3: **for each** generation g from 1 to G **do**
- 4: **for each** individual ($eps, MinPts$) **in** population P **do**
- 5: **Run** DBSCAN with ($eps, MinPts$) on dataset X
- 6: **Calculate** the following metrics:
- 7: **Intra-cluster Compactness**, f_1
- 8: **Inter-cluster Separation**, f_2
- 9: **Silhouette Coefficient**, f_3
- 10: **Combine** metrics using the weighted fitness function $f(x_i)$
- 11: **Add** $f(x_i)$ to S
- 12: **end for**
- 13: **Select** parents based on fitness scores S using roulette wheel selection
- 14: **Initialize** an empty list P_{new} for the next generation
- 15: **for each** pair of selected parents ($eps_1, MinPts_1$) and ($eps_2, MinPts_2$) **do**
- 16: **Perform** crossover to produce offspring ($eps_c1, MinPts_c1$) and ($eps_c2, MinPts_c2$)
- 17: **Apply** mutation to offspring with mutation rate r_{mut}
- 18: **Add** offspring to P_{new}
- 19: **end for**
- 20: **Update** population $P \leftarrow P_{new}$
- 21: **end for**
- 22: **Find** the best individual ($eps_{best}, MinPts_{best}$) with the highest $f(x_i)$
- 23: **Return** ($eps_{best}, MinPts_{best}$)

The State Grid traffic. We collected traffic in the platform tier control area of the distribution master station at the State Grid in Jining City from 10/1/2024 to 10/7/2024. We collected 2.39 TB packets in ten days. After preprocessing the raw traffic, we obtained 29.73 GB of traffic. We also injected eight types of attacks to simulate the real-world scenario: Web vulnerability scanning, network scanning, SQL injection, directory brute force, account discovery, account brute force, backdoor attacks, and other forms of malicious activity. We obtained the ground truth dataset with expert analysts manually labeling it by identifying malicious activities based on predefined indicators such as abnormal communication sequences, protocol misuse, and attack signatures. The labeling results were cross-validated using the Maltrail malicious traffic detection tool [29], which cross-referenced malicious traces from publicly available blocklists, antivirus reports, and custom user-defined lists, ensuring the accuracy and reliability of the labels. Table IV shows the details of real-world datasets.

TABLE IV
THE DETAILS OF REAL-WORLD DATASETS

Datasets	Description	Value
The CERNET traffic	Benign traffic	52.6%
	C&C	11.2%
	Https, SQL, TLS	9.1%
	Scan	8.2%
	TCP, UDP, DNS, NTP	9.7%
	Alerts, Logs	9.2%
	Total Sample Count	5,762,659
The State Grid traffic	Cleaned Sample Count	5,083,491
	Sliced Sample Count	962,593
	Benign traffic	51%
	Web Vulnerability Scanning	4.1%
	SQL Injection	5.7%
	Directory Brute Force	3.2%
	Account Discovery	10.1%
	Account Brute Force	12.6%
	Backdoor Attacks	13.3%
	Total Sample Count	4,806,341
	Cleaned Sample Count	4,283,562
	Sliced Sample Count	732,857

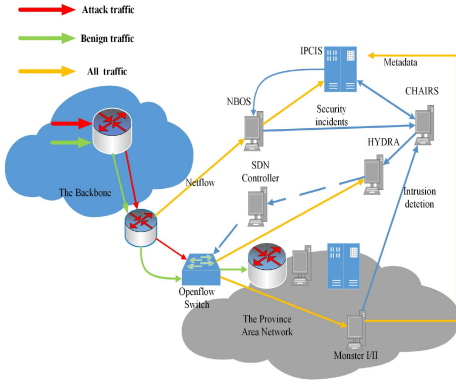


Fig. 3. The overall architecture of CERNET.

We use Recall, Accuracy, Precision and F1-Score to evaluate the performance and effectiveness of our algorithm. We also introduce the Adjusted Rand Index (ARI), the Fowlkes-Mallows Index(FMI), and Purity as the evaluation criteria [30]. The ARI score is a similarity metric between two clusters, which counts the number of data points correctly classified. A higher ARI value indicates a higher similarity of the clusters.

$$ARI = \frac{2 * (TP * TN - FN * FP)}{(TP + FN) * (FN + TN) + (TP + FP) * (FP + TN)} \quad (13)$$

FMI is the geometric mean of precision and recall with ranges from 0 to 1. It is a similarity metric between two clusters considering the number of TP , FP , TN , and FN . A higher score indicates a higher similarity between the clusters.

$$FMI = \sqrt{Precision * Recall} = \frac{TP}{\sqrt{(TP + FP) * (TP + FN)}} \quad (14)$$

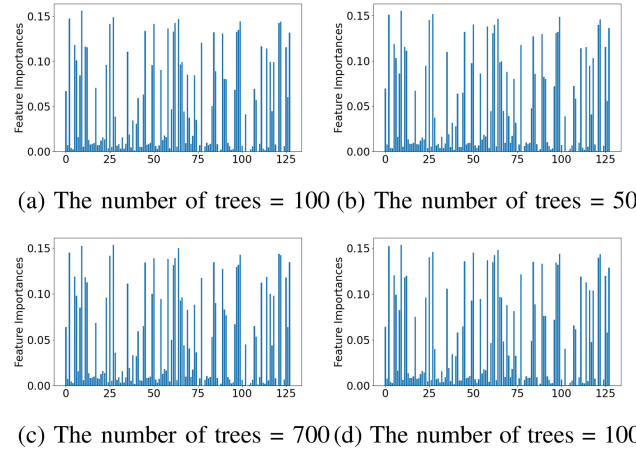


Fig. 4. The relative importance of each feature.

Purity refers to the percentage of correctly clustered samples compared to all samples during the clustering process. TP , FP , TN , and FN denote true positive, false positive, true negative, and false negative, respectively.

$$Purity(W, C) = \frac{1}{N} \sum_{i=1}^k (\max_j |w_i \cap c_j|) \quad (15)$$

where $W = \{ \omega_1, \omega_2, \dots, \omega_k \}$ is the set of clusters, $C = \{ c_1, c_2, \dots, c_k \}$ is the set of classes, and N refers to the total number of samples under clustering.

B. Performance Analysis of HyperEye

We conduct extensive experiments for performance evaluation in the section, including different cluster algorithm comparisons, feature subset selection, and system performance analysis.

Firstly, we applied the GBFS algorithm on our dataset with different tree values: 100, 500, 700, and 1000 to obtain the most promising features. The different tree values can assess how increasing the number of trees impacts feature importance and model performance. Fig. 4 shows that most tree values' top 58 features remained consistent.

Subsequently, we compared four classical optimization algorithms, e.g., the genetic algorithm (GA), the simulated annealing algorithm (SA), the ant colony optimization algorithm (ACO), and the particle swarm optimization algorithm (PSO), respectively. After several iterations, we found that the GA's runtime is the lowest, as shown in Fig. 5. Therefore, we chose GA to capture the best neighbourhood radius "eps" and the minimum core number "MinPts" of DBSCAN.

Then, we compare different cluster algorithms, including k-means, fuzzy c-means (FCM), spectral clustering (SC), hierarchical clustering (HC), Gaussian mixture model (GMM) models, DBSCAN, MCOSC [31], and our improved DBSCAN algorithm(GA-DBSCAN) with the fused characteristics to verify the superiority of our GA-DBSCAN. In k-means, a suitable k value range can fully demonstrate its performance. A low value of it will quickly assign different samples to the same cluster. However, a high one will cause much larger

TABLE V
CLUSTERING MODEL COMPARISONS UNDER DIFFERENT DATASETS

Algorithms	The CERNET traffic					CICIDS 2017				
	ARI	FMI	Purity	Accuracy	F1	ARI	FMI	Purity	Accuracy	F1
K-means	93.2%	94.1%	92.7%	93.2%	91.8%	92.8%	93.7%	93.2%	94.1%	93.6%
FCM	84.7%	86.1%	82.5%	85.7%	86.3%	85.2%	86.7%	83.5%	85.9%	86.2%
SC	89.3%	90.1%	79.8%	88.2%	88.7%	89.7%	91.5%	82.3%	89.7%	90.2%
HC	90.6%	91.7%	89.6%	91.6%	92.3%	91.2%	93.1%	90.1%	91.2%	92.7%
DBSCAN	97.3%	97.5%	97.9 %	98.1%	97.5%	97.5%	98.1%	98.2%	98.4%	97.7%
GA-DBSCAN	98.8%	99.1%	98.6%	99.6%	99.5%	98.9%	99.4%	98.9%	99.8%	99.7%
GMM	95.4%	95.8%	96.2%	94.5%	93.2%	95.7%	96.2%	96.8%	95.7%	95.2%
MCOSC[31]	98.3%	98.5%	98.1%	98.7%	98.6%	98.4%	98.7%	98.3%	98.5%	98.7%

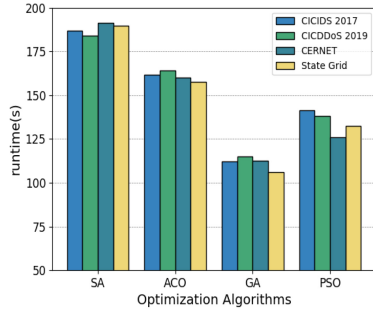


Fig. 5. The runtime of different optimization algorithms.

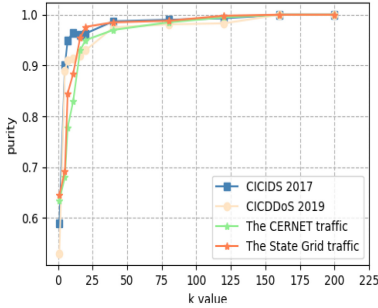


Fig. 6. Clustering purity with different k values.

clusters than actually. Therefore, A low value will affect the algorithm's accuracy, while a high one will affect its practicability.

Fig. 6 shows the purity of k-means under four different datasets. When k increases, the clustering purity naturally shows a steady growth trend. However, when it is above 153, the purity curves under different datasets slows and becomes stable. Therefore, we set the range of $k \in [5, 153]$ in the following experiments. The reason why k is equal to 5 is that we preconfigured five unknown traffic types.

After that, we compare our clustering model with the other seven clustering algorithms to show its advantages under different datasets. Table V shows each clustering algorithm's clustering purity, ARI score, FMI score, Overall Accuracy, and F1-score. We find that SC and FCM cluster algorithms show poor performance. The GA-DBSCAN algorithm obtains the highest clustering accuracy, while others show close detection accuracy.

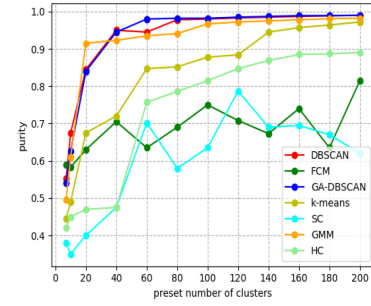


Fig. 7. Purity of different clustering algorithms under CERNET.

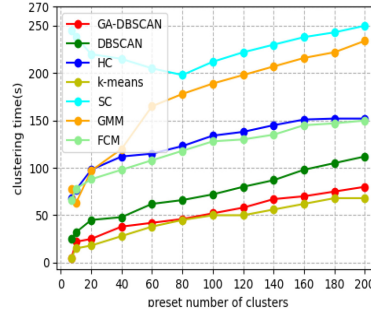


Fig. 8. Time consumption of different clustering algorithms under CERNET.

We also evaluate each clustering algorithm's purity and time consumption under the same number of clusters. Fig.7 shows that with the increased number of clusters, our improved DBSCAN algorithm obtains high clustering purity and achieves a high value after the number of clusters exceeds 60.

In addition, Fig. 8 shows their time consumption. In most cases, the clustering time increases with the number of clustering numbers. K-means obtain less time than other clustering algorithms. Although the time consumption of our GA-DBSCAN algorithm is bigger than k-means, the gap is insignificant. What is more, the purity, ARI score, FMI score, Overall Accuracy, and F1-score of our algorithm are better than it. Therefore, according to clustering purity and time consumption, our cluster algorithm is superior to other alternatives.

Finally, after evaluating our clustering algorithm, we verify the superiority of our feature-fusing algorithm under four

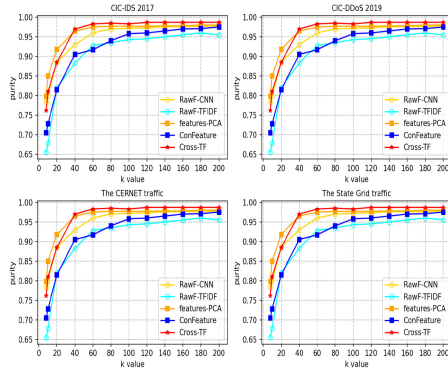


Fig. 9. The evaluation of feature-fusing algorithm under different datasets.

TABLE VI
COMPARISON WITH DIFFERENT ALGORITHMS UNDER THE CERNET TRAFFIC

Algorithms	ARI	FMI	Purity	Accuracy	F1-Score
Ours	99.1%	99.3%	99.4%	99.8%	99.6%
A4	96.8%	97.6%	97.7%	98.3%	98.7%
A3	95.8%	96.1%	97.2%	97.3%	97.8%
A1	93.5%	95.2%	94.3%	94.7%	95.1%
A2	90.1%	89.6%	90.7%	90.5%	91.2%
A5	84.6%	85.1%	85.3%	86.7%	87.4%

data sets. We have verified five types of features: raw single features extracted based on CNN (RawF-CNN), raw single features extracted based on TF-IDF (RawF-TFIDF), concatenated features by concatenating RawF-CNN and RawF-TFIDF (ConFeature), dimensionality reduction features based on PCA (features-PCA), and our cross-term fusion features (Cross-TF). We took them as input for our clustering algorithm in the experiment. Fig. 9 shows that the overall results of cross-TF and features-PCA are better than others, which proves that our cross-term fusion feature algorithm can effectively capture critical information.

C. Comparative Experiments

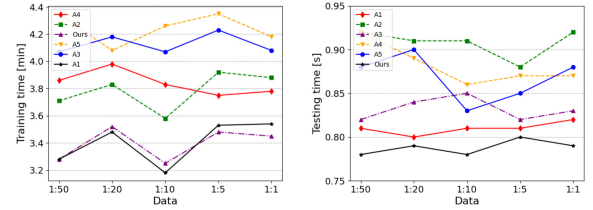
We use ARI, FMI, Purity, Accuracy, and F1 to verify the superiority and scalability of our model. State-of-the-art research focuses on these metrics and is a significant indicator of network traffic clustering. We use four datasets to evaluate the state-of-the-art works and verify the superiority of our model. They are [19], [23], [32], [33] and [15]. For convenience, we use “A1”, “A2”, “A3”, “A4”, and “A5” for short. Since their source code is not publicly available, we replicate their methodology with real-world traffic for normalization.

Since different parameters make it difficult to compare, we compute the highest value of clustering purity under different clustering numbers and choose the highest value for evaluation. When the cluster numbers equal to 63(Ours), 42(A2), 38(A1), 44(A5), 42(A3), and 35(A4), our algorithm outperforms other methods obviously, as shown in Tables VI and VII.

We also evaluated the time consumption of the different detection models. As the compared algorithms above used

TABLE VII
COMPARISON WITH DIFFERENT ALGORITHMS UNDER THE STATE GRID TRAFFIC

Algorithms	ARI	FMI	Purity	Accuracy	F1-Score
Ours	98.9%	99.1%	99.3%	99.7%	99.6%
A4	96.7%	97.4%	97.3%	97.5%	98.6%
A3	94.3%	95.5%	96.7%	96.3%	97.5%
A2	92.1%	94.5%	93.3%	94.0%	94.2%
A1	89.4%	89.1%	90.5%	90.3%	91.5%
A5	84.5%	85.4%	85.4%	87.5%	87.9%



(a) Training time

(b) Testing time

Fig. 10. The training and testing time of detection models.

TABLE VIII
PERFORMANCE EVALUATION AFTER ADDING NOISE DATA UNDER THE STATE GRID TRAFFIC

Algorithms	ARI	FMI	Purity	Accuracy	F1-Score
Ours	98.4%	98.8%	99.1%	99.4%	99.2%
A4	91.5%	92.3%	92.2%	92.4%	92.3%
A1	89.2%	89.4%	90.1%	89.8%	89.7%
A3	84.4%	84.2%	84.6%	85.3%	85.1%
A2	81.2%	81.6%	81.9%	81.4%	80.9%
A5	72.4%	72.2%	73.1%	73.6%	73.9%

different traffic traces, we replicated their methodology with CERNET traffic for normalization. The results are shown in Fig. 10. With the gradual balancing of data, there has been improvement in the detection performance across various methods. Although the training phase incurs some overhead, making our training time not the shortest, our cross-term fusion algorithm can reduce the feature dimensions by 32%, which enables efficient filtering of malicious traffic. Therefore, the memory overhead is significantly reduced.

Finally, we label-flip some of the malicious data as noise data mixed into benign traffic to test the robustness of our approach against evasion attacks. Table VIII shows the detection accuracy of the different methods after adding the noise data. Experimental results demonstrate that our method can effectively filter noise data and has excellent detection performance against evasion attacks.

V. CONCLUSION

In this paper, we propose HyperEye, a lightweight, real-time, unsupervised, encrypted malicious traffic detection algorithm that fuses the protocol-agnostic numerical and protocol-specific text features to identify unknown encrypted traffic. We convert protocol-agnostic numerical features to grey-scale maps and use 2D-CNN to capture their latent relationships. We use TF-IDF to transform the protocol-specific

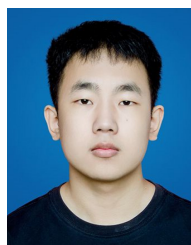
text features into vector representations and devise a cross-term fusion algorithm to obtain a comprehensive behavior description of traffic behavior. Finally, we design a genetic algorithm-based DBSCAN parameter optimization method to find the optimal parameter combination of clustering results automatically. We conducted extensive experiments, and the results show that HyperEye has 99.8% precision and 99.4% purity, outperforming other baseline methods.

REFERENCES

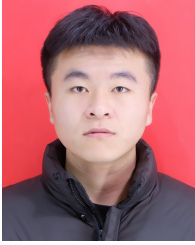
- [1] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1988–2014, 2nd Quart., 2019.
- [2] M. Husák, M. Čermák, T. Jirsík, and P. Čeleda, "HTTPS traffic analysis and client identification using passive ssl/tls fingerprinting," *EURASIP J. Inf. Security*, vol. 2016, pp. 1–14, Feb. 2016.
- [3] M. Villarreal-Vasquez, G. Modelo-Howard, S. Dube, and B. Bhargava, "Hunting for insider threats using LSTM-based anomaly detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 451–462, Jan./Feb. 2023.
- [4] Z. Deng, K. Chen, G. Meng, X. Zhang, K. Xu, and Y. Cheng, "Understanding real-world threats to deep learning models in android apps," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2022, pp. 785–799.
- [5] A. Lichy, O. Bader, R. Dubin, A. Dvir, and C. Hajaj, "When a RF beats a CNN and GRU, together—A comparison of deep learning and classical machine learning approaches for encrypted malware traffic classification," *Comput. Security*, vol. 124, Jan. 2023, Art. no. 103000.
- [6] C. Fu, Q. Li, and K. Xu, "Flow interaction graph analysis: Unknown encrypted malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 2972–2987, Mar. 2024.
- [7] Z. Fu et al., "Encrypted malware traffic detection via graph-based network analysis," in *Proc. 25th Int. Symp. Res. Attacks, Intrusions Defenses*, 2022, pp. 495–509.
- [8] Y. Hong, Q. Li, Y. Yang, and M. Shen, "Graph based encrypted malicious traffic detection with hybrid analysis of multi-view features," *Inf. Sci.*, vol. 644, Oct. 2023, Art. no. 119229.
- [9] X. Yang, S. Ruan, Y. Yue, and B. Sun, "PETNet: Plaintext-aware encrypted traffic detection network for identifying cobalt strike https traffics," *Comput. Netw.*, vol. 238, Jan. 2024, Art. no. 110120.
- [10] Z. Zeng, P. Xun, W. Peng, and B. Zhao, "Toward identifying malicious encrypted traffic with a causality detection system," *J. Inf. Security Appl.*, vol. 80, Feb. 2024, Art. no. 103644.
- [11] A. S. Shekhawat, F. D. Troia, and M. Stamp, "Feature analysis of encrypted malicious traffic," *Expert Syst. Appl.*, vol. 125, pp. 130–141, Jul. 2019.
- [12] Z. Wang and V. L. Thing, "Feature mining for encrypted malicious traffic detection with deep learning and other machine learning algorithms," *Comput. Security*, vol. 128, May 2023, Art. no. 103143.
- [13] Y. Javed, M. A. Khayat, A. A. Elghariani, and A. Ghafoor, "PRISM: A hierarchical intrusion detection architecture for large-scale cyber networks," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5070–5086, Nov./Dec. 2023.
- [14] "DISTILLER: Encrypted traffic classification via multimodal multitask deep learning," *J. Netw. Comput. Appl.*, vols. 183–184, Jun. 2021, Art. no. 102985.
- [15] K. Lin, X. Xu, and F. Xiao, "MFFusion: A multi-level features fusion model for malicious traffic detection based on deep learning," *Comput. Netw.*, vol. 202, Jan. 2022, Art. no. 108658.
- [16] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, "WF-Transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2024.
- [17] T. Shapira and Y. Shavitt, "FlowPic: Encrypted Internet traffic classification is as easy as image recognition," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2019, pp. 680–687.
- [18] Z. Liu, L. F. Zhang, and Y. Tang, "Enhancing malware detection for android apps: Detecting fine-granularity malicious components," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2023, pp. 1212–1224.
- [19] D.-J. Kim, N. G. B. Amma, and V. Sarveshwaran, "A novel split learning-based consumer electronics network traffic anomaly detection framework for smart city environment," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4197–4204, Feb. 2024.
- [20] G. Dogan, "ProTru: A provenance-based trust architecture for wireless sensor networks," *Int. J. Netw. Manag.*, vol. 26, no. 2, pp. 131–151, Mar. 2016.
- [21] G. D'Angelo, M. Ficco, and F. Palmieri, "Malware detection in mobile environments based on autoencoders and API-images," *J. Parallel Distrib. Comput.*, vol. 137, pp. 26–33, Mar. 2020.
- [22] H. Ma et al., "Deep-learning-based app sensitive behavior surveillance for android powered cyber-physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5840–5850, Aug. 2021.
- [23] D. Javeed, M. S. Saeed, I. Ahmad, P. Kumar, A. Jolfaei, and M. Tahir, "An intelligent intrusion detection system for smart consumer electronics network," *IEEE Trans. Consum. Electron.*, vol. 69, no. 4, pp. 906–913, Nov. 2023.
- [24] X. Ma, W. Zhu, J. Wei, Y. Jin, D. Gu, and R. Wang, "EETC: An extended encrypted traffic classification algorithm based on variant resnet network," *Comput. Security*, vol. 128, May 2023, Art. no. 103175.
- [25] A. Rukhin et al., *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, vol. 22, U.S. Dept. Commerce, Technol. Admin., Washington, DC, USA, 2001.
- [26] M. Kravchik and A. Shabtai, "Efficient cyber attack detection in industrial control systems using lightweight neural networks and PCA," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2179–2197, Jul./Aug. 2022.
- [27] H. Yang, Q. He, Z. Liu, and Q. Zhang, "Malicious encryption traffic detection based on NLP," *Security Commun. Netw.*, vol. 2021, pp. 1–10, Aug. 2021.
- [28] X. Zang, J. Gong, M. Wang, P. Gao, and G. Zhang, "IP traffic behavior characterization via semantic mining," *J. Netw. Comput. Appl.*, vol. 213, Apr. 2023, Art. no. 103603.
- [29] "Malicious traffic detection tool." Maltrail. 2024. [Online]. Available: <https://github.com/stamparm/maltrail>.
- [30] S. Zhao, Y. Zhang, and Y. Sang, "Towards unknown traffic identification via embeddings and deep autoencoders," in *Proc. 26th Int. Conf. Telecommun. (ICT)*, 2019, pp. 85–89.
- [31] L. Pan, Z. Han, Z. Shanshan, and W. Feng, "An optimal allocation method for power distribution network partitions based on improved spectral clustering algorithm," *Eng. Appl. Artif. Intell.*, vol. 123, Aug. 2023, Art. no. 106497.
- [32] J. Li, G. Cheng, Z. Chen, and P. Zhao, "Protocol clustering of unknown traffic based on embedding of protocol specification," *Comput. Security*, vol. 136, Jan. 2024, Art. no. 103575.
- [33] Y. Fu, X. Li, X. Li, S. Zhao, and F. Wang, "Clustering unknown network traffic with dual-path autoencoder," *Neural Comput. Appl.*, vol. 35, pp. 8955–8966, Jan. 2023.



Xiaodong Zang received the Ph.D. degree from the School of Cyber Science and Engineering, Southeast University, Nanjing, China, in 2020. He is a Postdoctoral Fellow with the College of Electronic Optical Engineering, Nanjing University of Posts and Telecommunications University, Nanjing. He is currently a Lecturer and a Master Supervisor with the School of Cyber Science of Engineering, Qufu Normal University. His research interests include computer networks and security, intrusion detection, network traffic, and host profiling.



Zilong Zheng received the B.S. degree in computer software from Qufu Normal University, Qufu, China, in 2020, where he is currently pursuing the M.D. degree. His current research focuses on computer networks and security and intrusion detection.



Haosheng Zheng received the B.S. degree in computer software from Qufu Normal University, Qufu, China, in 2024, where he is currently pursuing the M.D. degree. His current research focuses on computer networks and security, and intrusion detection.



Muhammad Khurram Khan (Senior Member, IEEE) is currently working as a Professor of Cybersecurity with the Center of Excellence in Information Assurance, King Saud University, Saudi Arabia. He is the Founder and the CEO of the 'Global Foundation for Cyber Studies and Research,' an independent and non-partisan cybersecurity think-tank in Washington, D.C, USA. His research areas of interest are cybersecurity, digital authentication, IoT security, biometrics, multimedia security, cloud computing security, cyber policy, and technological innovation management.



Xuan Liu (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from Southeast University, China, and he is currently a Lecturer and a Master Supervisor with the College of Information Engineering (College of Artificial Intelligence), Yangzhou University, China. His current research interests include network architecture, in-network computing, and cyberspace endogenous safety and security.



Weiwei Jiang received the B.Sc. degree in electronic engineering and the Ph.D. degree in information and communication engineering from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2013 and 2018, respectively. He is currently an Assistant Professor with the School of Information and Communication Engineering and the Key Laboratory of Universal Wireless Communications, Ministry of Education, Beijing University of Posts and Telecommunications. His current research interests include artificial intelligence, machine learning, big data, wireless communication, and edge computing.